

1 - Middleware & Authorization

We now have the ability to register users and authenticate. However, there is no purpose at the moment. Now we want to do a few things. First, we want to protect certain routes from guests and from auth users. We do this with something called middleware. So we'll learn about middleware. We also need to make it so that when we create a job listing, the user id from the session is put in the database with that job.

Then we can start to talk about policies, which allow us to authorize certain users to do certain actions. For instance, only the job listing owner should be able to update and delete that job. So this section is all about authorization and access control. One other thing I want to do is create another seeder that will quickly create a user for us that we can log in as and also make that user own a couple job listings.

2 - Middleware Overview

One of the most important parts of any web application is the middleware, which we have yet to really get into.

Middleware is a type of filter that sits between the HTTP request from the client and the application. It checks and processes incoming requests before they reach the controller or outgoing responses before they are sent back to the user.

For example, you can use middleware to verify that the user of your application is authenticated. If the user is not authenticated, the middleware will redirect the user to the login page. We are going to implement this but before we do that, I want to create some simple custom logging middleware.

Think of middleware as a gatekeeper for your application. It sits between the user and the application and decides whether the user is allowed to access the application.

Global vs Route Middleware

There are two types of middleware: global and route. Global middleware is run for every request, while route middleware is run for a specific route. I am going to show you both ways in this lesson.

Creating Middleware

Let's create a simple logging middleware that will log the request method and URI to the log file.

To create a middleware, we can use the `make:middleware` command:

```
php artisan make:middleware LogRequest
```

This command will create a new middleware class in the `app/Http/Middleware` directory. The class has one method called `handle`. This method is called when the middleware is run. It returns `$next($request);` by default, which means that it will pass the request on to the next middleware in the chain.

Next, let's open the `app/Http/Middleware/LogRequest.php` file and just add a simple print for now:

```
public function handle(Request $request, Closure $next): Response
{
    print('From the LogRequest middleware');
    return $next($request);
}
```

Registering Global Middleware

To register the middleware to run globally on all routes, we need to add it to the `$routeMiddleware` array in the `bootstrap/app.php` file. Open that file and import the middleware:

```
use App\Http\Middleware\LogRequest;
```

Then add it to the `->withMiddleware()` closure:

```
->withMiddleware(function (Middleware $middleware) {
    $middleware->append(LogRequest::class);
})
```

The `append` method adds the middleware to the end of the list of global middleware. If you would like to add a middleware to the beginning of the list, you should use the `prepend` method.

Now when you go to any route, you will see the print statement.

Log Requests

Instead of printing, let's use the `Log` facade to log the request method and URI to the log file. Open the `app/Http/Middleware/LogRequest.php` file and replace the `handle` method with the following:

```
public function handle(Request $request, Closure $next): Response
{
    Log::info("{ $request->method()} - { $request->fullUrl()}");
    return $next($request);
}
```

Now your requests will be logged to the `storage/logs/laravel.log` file. Open it up and go to the end of the file and you should see something like the following:

```
[2024-08-19 11:15:38] local.INFO: GET - http://127.0.0.1:8000
[2024-08-19 11:15:41] local.INFO: GET - http://127.0.0.1:8000/jobs
[2024-08-19 11:16:01] local.INFO: GET - http://127.0.0.1:8000/jobs/1
```

Later on, we will learn more about logging. I will show you how to create an artisan command to clear the log file.

Assigning Middleware to Routes

So this is global middleware. Let's say you want this to only run on a specific route. You can do that by passing the middleware to the route.

First, delete or comment out the following line in the `bootstrap/app.php` file:

```
->withMiddleware(function (Middleware $middleware) {  
    // $middleware->append(LogRequest::class);  
})
```

Now open the `routes/web.php` file and add the following import:

```
use App\Http\Middleware\LogRequest;
```

Then add the middleware to the home route:

```
Route::get('/', [HomeController::class, 'index'])->name('home')->  
>middleware(LogRequest::class);
```

Now you will only see logs from the home route. This is a bit pointless, but it shows you how to use middleware.

Go ahead and remove the middleware from the home route. You can delete the `LogRequest` middleware if you want.

3 - Auth Middleware & Protecting Routes

In Laravel, you can protect routes by using the `auth` middleware. As we saw in the last lesson, middleware is a way to filter HTTP requests entering your application.

Manually Checking User Authentication

You can still check if a user is authenticated manually by using the `Auth` facade. Let's say that you don't want users to be able to visit the `/jobs.create` route unless they are logged in. Right now, sure the button doesn't show, but they can still visit the route as a guest.

To protect the route manually, you can go into the `JobsController create` method and add the following:

Add the import:

```
use Illuminate\Support\Facades\Auth;
```

Then add the following to the `create` method. Also add `View | RedirectResponse` to the method signature.

```
public function create(): View | RedirectResponse
{
    if (!Auth::check()) {
        return redirect()->route('login');
    }

    return view('jobs.create');
}
```

Now if you try to visit the `/jobs.create` route, you'll be redirected to the login page.

This is ok, but what if you want to protect multiple routes? You could copy and paste the code into each method, but that's not very DRY. Instead, you can use middleware. Delete the code we just added.

Using Middleware

Laravel comes with a built-in middleware for checking if a user is authenticated. It's called `auth`. You can use this middleware to protect routes.

To use the `auth` middleware, you can add it to the route definition in the `web.php` file.

Let's say that we want only logged in users to access the homepage, we would add the following to the `web.php` file:

```
Route::get('/', [HomeController::class, 'index']->name('home')-
>middleware('auth'));
```

Simple right? However, remember that we are using resource routes of jobs. We want to apply the `auth` middleware only to the create, store, edit, destroy and update routes. To do this, you can use the `only` method:

```
// Apply middleware to specific actions
Route::resource('jobs', JobController::class)->middleware('auth')->only(['create',
'edit', 'destroy']);
```

Now you will get an error like `jobs.show is not defined`. This is because we are using resource routes. To fix this, you can use the `except` method. Add this right under the code you just added:

```
// Define the rest of the resource routes without middleware
Route::resource('jobs', JobController::class)->except(['create', 'edit',
'destroy']);
```

Now you can visit the `/jobs.create` route and you will be redirected to the login page.

Next, we will look at the `guest` middleware as well as middleware groups.

4 - Guest Middleware & Groups

Now that we have protected routes like `/jobs/create` and `/jobs/edit/:id`, I now want to use the `Guest` middleware to make it so only guests or non-logged in users can access routes like `/login` and `/register`.

Open the `routes/web.php` file and add the following to the login route:

```
Route::get('/login', [LoginController::class, 'login'])->name('login')-
>middleware('guest')->middleware('guest');
```

This is fine and we can add it to the register route as well. However, we can use a `group` to apply the middleware to multiple routes.

```
Route::middleware('guest')->group(function () {
    Route::get('/register', [RegisterController::class, 'register'])->
>name('register');
    Route::post('/register', [RegisterController::class, 'store'])->
>name('register.store');
    Route::get('/login', [LoginController::class, 'login'])->name('login');
    Route::post('/login', [LoginController::class, 'authenticate'])->
>name('login.authenticate');
});
```

This is a bit cleaner. Now, only guests can access the login and register routes. If a user is logged in and tries to access these routes, they will be redirected to the home page.

5 - Test User Seeder

We already have the ability to wipe the database jobs and users and recreate 10 new ones with the following command:

```
php artisan db:seed
```

I also want it to create a test user so that I don't have to keep registering a new user every time I want to test something.

Let's create a new seeder:

```
php artisan make:seeder TestUserSeeder
```

Open the file `database/seaders/TestUserSeeder.php` and add the following code:

```
<?php

namespace Database\Seeders;

use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\Hash;
use App\Models\User;
use Carbon\Carbon;

class TestUserSeeder extends Seeder
{
    /**
     * Run the database seeds.
     */
    public function run(): void
    {
        User::create([
            'name' => 'Test User',
            'email' => 'test@test.com',
            'email_verified_at' => Carbon::now(),
            'password' => Hash::make('12345678'),
        ]);
    }
}
```

Now we need to add the seeder to the `DatabaseSeeder` class. Open the file `database/seaders/DatabaseSeeder.php` and add the line to call the new seeder:

```
public function run(): void
{
    // Truncate tables
    DB::table('job_listings')->truncate();
    DB::table('users')->truncate();

    $this->call(TestUserSeeder::class); // Add this line
    $this->call(RandomUserSeeder::class);
    $this->call(JobSeeder::class);
}
```

Assign the User ID To Listings

Let's make it so that a couple of the listings are created by the test user. Open the file `database/seeder/JobSeeder.php` and change the `run` method to the following:

```
public function run(): void
{
    // Load job listings data
    $jobListings = include database_path('seeders/data/job_listings.php');

    // Get the ID of the user created by TestUserSeeder
    $testUserId = User::where('email', 'test@test.com')->value('id');

    // Get all other user IDs
    $userIds = User::where('email', '!=', 'test@test.com')->pluck('id')->toArray();

    foreach ($jobListings as $index => &$listing) {
        if ($index < 2) {
            // Assign the first two job listings to the test user
            $listing['user_id'] = $testUserId;
        } else {
            // Assign the rest to random users
            $listing['user_id'] = $userIds[array_rand($userIds)];
        }
        // Add timestamps
        $listing['created_at'] = now();
        $listing['updated_at'] = now();
    }

    // Insert job listings
    DB::table('job_listings')->insert($jobListings);
}
```

We get the test user ID and all other user IDs. Then we loop through the job listings and assign the first two to the test user and the rest to random users.

Now every time we run `php artisan db:seed`, it will create a new user with the email `test@test.com` and the password `12345678`. It will also create 10 new job listings, with the first two being created by the test user and the rest being created by random users.

6 - Add Current User To Listing

We have full authentication implemented in our application. However, we still need to implement authorization. Right now, anyone can edit or delete any listing. We need to implement authorization so that only the user who created a listing can edit or delete it.

Add User ID When Creating a Listing

The first step is to add a user ID to the listing when it is created. If you open the file `/app/Http/Controllers/JobController.php`, you will see that the `store` method has this line:

```
// Add the hardcoded user_id
$validatedData['user_id'] = 1;
```

This line is hardcoded to the user ID of 1. We need to change this to the user ID of the currently authenticated user. We can do this by adding the following line to the `store` method:

```
// Add the user ID of the current user
$validatedData['user_id'] = auth()->user()->id;
```

Now, log in and create a new job listing. If you check the database, you will see that the user ID of the listing is the user ID of the currently authenticated user.

Add Authorization to the Edit and Delete Buttons

Right now, the edit and delete buttons on the listings are always visible. Let's make it so the user has to be logged in and the user who created the listing can see the edit and delete buttons. In the next couple videos we are going to simplify this a bit by creating something called a policy and using a directive called `@can`. But for now, we will just do a manual condition.

Open the file `/resources/views/jobs/show.blade.php` and change the edit and delete buttons to the following:

```
@auth @if (auth()->user()->id === $job->user_id)
<div class="flex space-x-3 ml-4">
  <a
    href="{{ route('jobs.edit', $job->id) }}"
    class="px-4 py-2 bg-blue-500 hover:bg-blue-600 text-white rounded"
    >Edit</a>
  <!-- Delete Form -->
  <form
    method="POST"
    action="{{ route('jobs.destroy', $job->id) }}"
    onsubmit="return confirm('Are you sure you want to delete this job?');"
  >
    @csrf @method('DELETE')
    <button
      type="submit"
      class="px-4 py-2 bg-red-500 hover:bg-red-600 text-white rounded"
    >
      Delete
    </button>
  </form>
<!-- End Delete Form -->
```

```
</div>
@endif @endauth ````
```

We are checking if the user is logged in and if the user ID of the currently authenticated user is the same as the user ID of the listing. If so, we show the edit and delete buttons.

This is fine but in the next lesson, I want to show you how to use the `@can` directive to do this by creating what we call a "policy".

7 - Policies & `@can` Directive

In the last lesson, we added authorization to the edit and delete buttons on the job listings. We can still do the actual update and delete without owning the listing, which we need to change, but first I want to show you how to create a new policy and use the `@can` directive.

Open the terminal and run the following command to create a new policy:

```
php artisan make:policy JobPolicy --model=Job
```

This will create a file at `app/Policies/JobPolicy.php`. Open this file and you will see methods like `viewAny`, `view`, `create`, `update`, `delete`, etc. This is where you can define the authorization rules for the `Job` model.

let's add the following code to the `update` method:

```
public function update(User $user, Job $job)
{
    return $user->id === $job->user_id;
}
```

This method will return `true` if the user ID of the currently authenticated user is the same as the user ID of the job listing. If it returns `true`, the user can update the job listing. If it returns `false`, the user cannot update the job listing.

Let's add the following code to the `delete` method:

```
public function delete(User $user, Job $job)
{
    return $user->id === $job->user_id;
}
```

This will do the same as the `update` method, but for the `delete` method.

Register the Policy

We need to register this policy within an auth service provider. Let's create a new auth service provider by running the following command:

```
php artisan make:provider AuthServiceProvider
```

A service provider is a class that registers bindings in the service container. It contains a `boot` method that is called when the application is booted. This is where we can register our policies.

Open the file `app/Providers/AuthServiceProvider.php` and add the following imports:

```
use App\Models\Job;
use App\Policies\JobPolicy;
use Illuminate\Foundation\Support\Providers\AuthServiceProvider as
ServiceProvider;
```

Remove this import:

```
use Illuminate\Support\ServiceProvider;
```

This may be a bit confusing, so let me explain why we switched these imports. When you generate a service provider in Laravel the default class that gets extended is that `Illuminate\Support\ServiceProvider`. This is because service providers are typically meant to be used to register services, bindings, etc within the service container, which is the primary purpose of the `ServiceProvider` class.

However, for certain types of service providers, like `AuthServiceProvider`, Laravel provides specialized base classes such as this `Illuminate\Foundation\Support\Providers\AuthServiceProvider` that include additional functionality specific to that domain, such as the `registerPolicies` method, which is what we need to call. That method is not available in the original `Illuminate\Support\ServiceProvider` class.

Add the following property above the `register` method:

```
protected $policies = [
    Job::class => JobPolicy::class,
];
```

The `$policies` array is used to define which policy class should be used for a specific Eloquent model. In this case, it maps the `Job` model (`Job::class`) to the `JobPolicy` class (`JobPolicy::class`).

Then, add the following code to the `boot` method:

```
public function boot()
{
    $this->registerPolicies();
}
```

```
}
```

This will register the policies.

Use the `@can` Directive

Now that we have a policy, we can use the `@can` directive to check if the user can update or delete the job listing. Open the file `resources/views/jobs/show.blade.php` and change the edit and delete buttons to the following:

```
@can('update', $job)
<div class="flex space-x-3 ml-4">
  <a
    href="{{ route('jobs.edit', $job->id) }}"
    class="px-4 py-2 bg-blue-500 hover:bg-blue-600 text-white rounded"
  >Edit</a>
  <!-- Delete Form -->
  <form
    method="POST"
    action="{{ route('jobs.destroy', $job->id) }}"
    onsubmit="return confirm('Are you sure you want to delete this job?');"
  >
    @csrf @method('DELETE')
    <button
      type="submit"
      class="px-4 py-2 bg-red-500 hover:bg-red-600 text-white rounded"
    >
      Delete
    </button>
  </form>
<!-- End Delete Form -->
</div>
@endcan
```

This is a lot cleaner than the previous code. We can use the `@can` directive to check if the user can update or delete the job listing.

We still need to stop the actual update and delete without owning the listing, which we need to change. We will do this in the next lesson.

8 - Policy & Authorization In Controller

Right now, the policy is only preventing the user from seeing the edit and delete buttons. We need to use the policy in the job controller so they actually can't update or delete unless they own the listing.

Open the file `app/Http/Controllers/JobController.php` and add the following import:

```
use Illuminate\Foundation\Auth\Access\AuthorizesRequests;
```

We also need to add the following to the class:

```
class JobController extends Controller
{
    use AuthorizesRequests;

    // ...rest of the class
}
```

This will make the `AuthorizesRequests` trait available to the class. So we can use methods like `authorize` and `authorizeForUser` in the controller.

Now, let's add the following code to the `update` method:

```
public function update(Request $request, Job $job): RedirectResponse
{
    // Check if the user is authorized
    $this->authorize('update', $job);

    // ...rest of the method
}
```

Add the following code to the `destroy` method:

```
public function destroy(Job $job): RedirectResponse
{
    // Check if the user is authorized
    $this->authorize('delete', $job);
    // ...rest of the method
}
```

Now if you try and edit or delete a listing that you don't own, you will get a 403 error.

Prevent User From Seeing The Edit Form

We also need to prevent the user from seeing the edit form if they don't own the listing. Open the file `app/Http/Controllers/JobController.php` and add the following code to the `edit` method:

```
public function edit(Job $job): View
{
    // Check if the user is authorized
```

```
$this->authorize('update', $job);  
// ...rest of the method  
}
```

Now they can not even see the form if they don't own the listing.

That is it as far as the authorization goes for job listings.